Bandersnatch VRF-AD Specification

Davide Galassi Seyed Hosseini

20 Oct 2026 - Draft 36

Abstract

This specification defines three Verifiable Random Function with Additional Data (VRF-AD) schemes – Tiny VRF, Thin VRF, and Pedersen VRF – built on a transcript-based Fiat-Shamir transform with support for multiple input/output pairs via delinearization. Tiny VRF and Thin VRF are loosely inspired by IETF ECVRF RFC-9381 [1], adapted with a transcript-based Fiat-Shamir transform, support for additional data, and multiple I/O pairs via delinearization. Pedersen VRF follows the construction introduced by BCHSV23 [2] and serves as a building block for anonymized ring signatures based on the ring proof scheme derived from CSSV22 [3]. All schemes are instantiated over the Bandersnatch elliptic curve, constructed over the BLS12-381 scalar field as specified in MSZ21 [4].

1. Preliminaries

1.1. Groups and Fields

- $\mathbf{G}$: Bandersnatch curve cyclic group of prime order r, defined over the base field of prime order q.
- $\mathbf{F}$: Scalar field of prime order r (i.e. $\mathbb{Z}_r$).
- $\mathbf{\Sigma}^k$: Octet strings with length $k \in \mathbb{N}$ ($*$ for arbitrary length).
- $\mathcal{O}$: Identity point of $\mathbf{G}$.

The EC group $\mathbf{G}$ is the prime subgroup of the Bandersnatch elliptic curve, in Twisted Edwards form, with finite field and curve parameters as specified in MSZ21 [4]. For this group, `fLen` = `qLen` = 32 and `cofactor` = 4.

All point arithmetic MUST be performed in $\mathbf{G}$. Points not in $\mathbf{G}$ MUST be rejected at all entry points; accepting a point on the full curve but outside the prime-order subgroup enables small-subgroup attacks that break the VRF relation.

1.2. Notation

- $x \in \mathbf{F}$: Secret key scalar.
- $Y \in \mathbf{G}$: Public key point defined as $x \cdot G$.
- $i \in \mathbf{\Sigma}^*$: VRF input data.
- $I \in \mathbf{G}$: VRF input point.
- $O \in \mathbf{G}$: VRF output point.
- $o \in \mathbf{\Sigma}^k$: VRF output hash.
- T: Transcript state.

The *group generator* $G \in \mathbf{G}$ is defined as:

$x = 18886178867200960497001835917649091219057080094937609519140440539760939937304$
$y = 19188667384257783945677642223292697773471335439753913231509108946878080696678$

1.3. VRF-AD

Regardless of the specific scheme, a *Verifiable Random Function with Additional Data (VRF-AD)* can be concisely represented by three primary functions:

Abstract interface:

- `prove`$(x \in \mathbf{F}, \overline{io} \in (\mathbf{G} \times \mathbf{G})^n, ad \in \mathbf{\Sigma}^*) \to \Pi$
- `verify`$(Y \in \mathbf{G}, \overline{io} \in (\mathbf{G} \times \mathbf{G})^n, ad \in \mathbf{\Sigma}^*, \Pi) \to (\top \mid \bot)$
- `output`$(O \in \mathbf{G}) \to o \in \mathbf{\Sigma}^N$

For Pedersen VRF (section 4), the public key Y is not an explicit input to `verify`; the proof Π carries a blinded commitment $\bar{Y}$ instead.

The additional data ad is an arbitrary-length octet-string signed together with the VRF output. It does not influence the produced VRF output. The length of ad MUST NOT exceed $2^{64} - 1$ bytes, as the length is encoded via `enc_64` in the VRF transcript (section 1.6.5).

1.4. Constants

- `suite_id` = `"Bandersnatch-SHA512-ELL2-v1"` — the 27-byte ASCII string identifying the cipher suite. It bundles several tightly-coupled choices: curve (Bandersnatch in Twisted Edwards form), transcript construction (HashTranscript over SHA-512), nonce algorithm (RFC-8032 inspired), challenge derivation (transcript squeeze), point encoding (compressed little-endian), hash-to-curve (Elligator 2 random oracle), and security level (128-bit). Bump the trailing version suffix when any of these changes. Version 2 of this suite, `"Bandersnatch-SHA512-ELL2-v2"`, differs only in the Z_{pad} prefix of hash-to-curve (Appendix A.2), which follows RFC-9380

exactly. The reference implementation provides it as `bandersnatch_v2`. It may replace this version in a future draft.

- `challenge_len` = 16 bytes (128-bit security).
- `expanded_scalar_len` = $\lceil(\lceil\log_2(r)\rceil + 128)/8\rceil$ = 48 bytes.

Domain separation tags used throughout the protocol:

Tag	Value	Usage
TinyVrf	0x00	Tiny VRF scheme identifier
ThinVrf	0x01	Thin VRF scheme identifier
PedersenVrf	0x02	Pedersen VRF scheme identifier
NonceExpand	0x10	Nonce secret expansion
Nonce	0x11	Nonce derivation
PedersenBlinding	0x12	Pedersen blinding factor
PointToHash	0x20	VRF output hashing
Delinearize	0x30	Delinearization scalars
Challenge	0x40	Challenge derivation
BatchVerify	0x50	Batch verification randomization
HashToCurve	0x60	Hash-to-curve domain separation

1.5. Codec

- `enc_scalar`($s \in \mathbf{F}$): Encodes a scalar into 32 octets in little-endian representation.
- `dec_scalar`($buf \in \mathbf{\Sigma}^{32}$): Interpret octet string buf as a little-endian integer. MUST output "INVALID" if the resulting value is $\geq r$.
- `enc_point`($P \in \mathbf{G}$): Encodes a point in compressed form. The y coordinate is serialized in little-endian and the most significant bit of the last octet encodes the sign of the x-coordinate. This gives `ptLen` = `fLen` = 32.
- `dec_point`($buf \in \mathbf{\Sigma}^{32}$): Interpret octet string buf as a compressed point. Mask the sign bit from the last octet and interpret the result as a little-endian integer. MUST output "INVALID" if the resulting value is $\geq q$. Otherwise, decompress the point and MUST output "INVALID" if it does not decode to a point on the prime subgroup $\mathbf{G}$.
- `dec_scalar_mod`($buf \in \mathbf{\Sigma}^*$): Interpret octet string buf as a little-endian integer and reduce modulo the prime field order r.
- `enc_64`($n \in \mathbb{N}_{2^{64}}$): Encode integer n as an 8-byte little-endian octet string.

Aggregate types (e.g. proofs) MUST be encoded as the concatenation of their individual fields in the order given by the structure definition, without any separator.

1.6. Procedures

1.6.1. Transcript

The transcript provides a Fiat-Shamir transform with an absorb/squeeze interface. Data is absorbed into an internal hash state; output bytes are squeezed from it. After the first squeeze, `absorb` MUST NOT be called.

Abstract interface:

- `new_transcript`() $\to T$: Create a fresh transcript instance and absorb `suite_id`.
- `absorb`($data \in \mathbf{\Sigma}^*$): Feed bytes into the hash state. MUST NOT be called after squeeze.
- `squeeze`($n \in \mathbb{N}$) $\to \mathbf{\Sigma}^n$: Produce n output bytes.
- `fork`() $\to T$: Clone the transcript state.

A concrete instantiation using SHA-512 is given in Appendix A.1.

1.6.2. VRF Input

The VRF input point $I \in \mathbf{G}$ is derived from the input octet-string using a `hash_to_curve` function that maps arbitrary-length octet-strings to points in $\mathbf{G}$.

$$I \leftarrow \texttt{hash_to_curve}(i)$$

The function MUST behave as a random oracle: its output must be indistinguishable from a uniformly random point in $\mathbf{G}$, and the discrete logarithm of the output with respect to any known base must be unknown.

Verifiers MUST independently compute each I_i from the corresponding input octet-string using the procedure above. Accepting prover-supplied input points without recomputation breaks the VRF security guarantees, and in the case of Thin VRF (section 3), enables trivial forgery.

A concrete instantiation using Elligator 2 is given in Appendix A.2.

1.6.3. VRF Output

The VRF output point is generated from the VRF input point and secret key scalar:

$$O \leftarrow x \cdot I$$

The VRF output hash is a fixed-length octet string derived from the output point using a transcript-based point-to-hash procedure. The procedure is deliberately

independent of the proof scheme: for a given key and input, the output point $O = x \cdot I$ is unique regardless of whether Tiny VRF, Thin VRF, or Pedersen VRF is used to prove correctness. The scheme determines how the proof is constructed, not the VRF output itself. This separation allows applications to obtain consistent output hashes across schemes for the same underlying evaluation.

Input:

- $O \in \mathbf{G}$: VRF output point.
- $N \in \mathbb{N}$: Desired output length in bytes. MUST be fixed per application context; N is not absorbed into the transcript, so $\texttt{squeeze}(N_1)$ is a prefix of $\texttt{squeeze}(N_2)$ for $N_1 < N_2$.

Output:

- $o \in \mathbf{\Sigma}^N$: VRF output hash.

Steps:

1. $T \leftarrow \texttt{new_transcript}()$
2. $T.\texttt{absorb}(\texttt{PointToHash} \parallel \texttt{enc_point}(O))$
3. $o \leftarrow T.\texttt{squeeze}(N)$

Transcript:

- $T = \texttt{suite_id} \parallel \texttt{PointToHash} \parallel \texttt{enc_point}(O)$

1.6.4. Delinearization

Merges input/output pairs into a single pair using delinearization scalars derived from the transcript. For $n = 0$, returns the identity pair. For $n = 1$, the pair is returned unchanged ($z_0 = 1$). For $n \geq 2$, random scalars prevent an attacker from mixing components across pairs.

Input:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: Sequence of input/output pairs.
- T: Transcript state.

Output:

- $(I_m, O_m) \in \mathbf{G} \times \mathbf{G}$: Merged input/output pair.

Steps:

1. If $n = 0$: return $(\mathcal{O}, \mathcal{O})$
2. $T.\texttt{absorb}(\texttt{Delinearize})$
3. $z_0 \leftarrow 1$

4. For $i = 1, \ldots, n-1$: $z_i \leftarrow \texttt{dec_scalar_mod}(T.\texttt{squeeze}(\texttt{challenge_len}))$
5. $I_m \leftarrow \sum_{i=0}^{n-1} z_i \cdot I_i,\ O_m \leftarrow \sum_{i=0}^{n-1} z_i \cdot O_i$
6. Return (I_m, O_m)

Transcript (where T_{in} is the caller-supplied state):

- $T = T_{in} \parallel \texttt{Delinearize}$

1.6.5. VRF Transcript

Shared transcript construction used by all VRF-AD schemes. Absorbs input/output pairs, merges them via delinearization (section 1.6.4), and absorbs additional data.

Input:

- *scheme*: Scheme identifier tag.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: Sequence of input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

Output:

- T: Transcript state.
- $(I_m, O_m) \in \mathbf{G} \times \mathbf{G}$: Merged input/output pair.

Steps:

1. $T \leftarrow \texttt{new_transcript}()$
2. $T.\texttt{absorb}(\textit{scheme})$
3. $T.\texttt{absorb}(\texttt{enc_64}(n))$
4. For each (I_i, O_i) in $\overline{io}$: $T.\texttt{absorb}(\texttt{enc_point}(I_i) \parallel \texttt{enc_point}(O_i))$
5. $T.\texttt{absorb}(\texttt{enc_64}(\texttt{len}(ad)) \parallel ad)$
6. $(I_m, O_m) \leftarrow \texttt{delinearize}(\overline{io}, T.\texttt{fork}())$
7. Return $(T, (I_m, O_m))$

Transcript:

$T = \texttt{suite_id} \parallel \textit{scheme}$
$\parallel \texttt{enc_64}(n) \parallel \texttt{enc_point}(I_0) \parallel \texttt{enc_point}(O_0) \parallel \cdots \parallel \texttt{enc_point}(I_{n-1}) \parallel \texttt{enc_point}(O_{n-1})$
$\parallel \texttt{enc_64}(\texttt{len}(ad)) \parallel ad$

1.6.6. Nonce

Deterministic nonce generation inspired by [RFC-8032] section 5.1.6. The transcript carries shared state from `vrf_transcript`, binding the nonce to the I/O pairs and additional data.

Input:

- $d \in \mathbf{F}$: Secret scalar.
- T: Transcript state.

Output:

- $k \in \mathbf{F}$: Nonce scalar.

Steps:

1. $T' \leftarrow T.\texttt{fork}()$
2. $T'.\texttt{absorb}(\texttt{NonceExpand} \parallel \texttt{enc_scalar}(d))$
3. $h \leftarrow T'.\texttt{squeeze}(64)$
4. $T.\texttt{absorb}(\texttt{Nonce} \parallel h)$
5. $k \leftarrow \texttt{dec_scalar_mod}(T.\texttt{squeeze}(\text{expanded_scalar_len}))$
6. If $k = 0$: abort (implementation error; probability $\approx 2^{-253}$).

Transcript (where T_{in} is the caller-supplied state):

- $T' = T_{in} \parallel \texttt{NonceExpand} \parallel \texttt{enc_scalar}(d)$
- $T = T_{in} \parallel \texttt{Nonce} \parallel h$

1.6.7. Challenge

Derives a challenge scalar by absorbing curve points into the transcript and squeezing.

Input:

- $\bar{P} \in \mathbf{G}^m$: Sequence of m points.
- T: Transcript state.

Output:

- $c \in \mathbf{F}$: Challenge scalar.

Steps:

1. $T.\texttt{absorb}(\texttt{Challenge})$
2. For each P_i in $\bar{P}$: $T.\texttt{absorb}(\texttt{enc_point}(P_i))$
3. $c \leftarrow \texttt{dec_scalar_mod}(T.\texttt{squeeze}(\texttt{challenge_len}))$

Transcript (where T_{in} is the caller-supplied state):

- $T = T_{in} \parallel \texttt{Challenge} \parallel \texttt{enc_point}(P_0) \parallel \cdots \parallel \texttt{enc_point}(P_{m-1})$

2. Tiny VRF

Compact VRF-AD scheme producing a short (c, s) proof. Like Thin VRF, it prepends the Schnorr pair (G, Y) to the I/O list and proves a single DLEQ on the delinearized merged pair. The challenge scalar c is stored instead of the nonce commitment, yielding a smaller proof at the cost of not supporting batch verification.

Security: VRF input points MUST be constructed via hash-to-curve. If a prover knows d such that $I = d \cdot G$, they can forge arbitrary outputs for that input, because the delinearization merges the Schnorr and VRF pairs into a single check that collapses when all points are multiples of G.

Proof encoding: The challenge c is produced by squeezing `challenge_len` bytes from the transcript. Since $2^{8 \cdot \texttt{challenge_len}} < r$, no modular reduction occurs and c is encoded as its raw `challenge_len`-byte little-endian representation. The scalar s is encoded via `enc_scalar` (32 bytes). The total proof size is `challenge_len` + 32 bytes. Verifiers MUST reject proofs where $c \geq 2^{8 \cdot \texttt{challenge_len}}$.

2.1. Prove

Input:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

Output:

- $\pi = (c, s) \in (\mathbf{F}, \mathbf{F})$: Schnorr-like proof.

Steps:

1. $Y \leftarrow x \cdot G$
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{TinyVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}())$
4. $R \leftarrow k \cdot I_m$
5. $c \leftarrow \texttt{challenge}([R], T)$
6. $s \leftarrow k + c \cdot x$
7. $\pi \leftarrow (c, s)$

2.2. Verify

Input:

- $Y \in \mathbf{G}$: Public key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (c, s) \in (\mathbf{F}, \mathbf{F})$: Schnorr-like proof.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

Steps:

1. Validate Y and all $I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\bot$ if any is invalid or the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{TinyVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $R \leftarrow s \cdot I_m - c \cdot O_m$
4. $c' \leftarrow \texttt{challenge}([R], T)$
5. $\theta \leftarrow \top$ if $c = c'$ else $\bot$

3. Thin VRF

Thin VRF is structurally similar to Tiny VRF: it prepends (G, Y) to the I/O pairs, applies delinearization, and proves a single DLEQ on the merged pair. The difference is the proof format: Thin VRF stores the nonce commitment R rather than the challenge c, which enables batch verification at the cost of a slightly larger proof.

3.1. Prove

Input:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

Output:

- $\pi = (R, s) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

Steps:

1. $Y \leftarrow x \cdot G$
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y)] \parallel \overline{io}, ad)$

3. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}())$
4. $R \leftarrow k \cdot I_m$
5. $c \leftarrow \texttt{challenge}([R], T)$
6. $s \leftarrow k + c \cdot x$
7. $\pi \leftarrow (R, s)$

3.2. Verify

Input:

- $Y \in \mathbf{G}$: Public key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (R, s) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

Steps:

1. Validate $R \in \mathbf{G}$ and $Y, I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$ for all i, output $\bot$ if any point is not in $\mathbf{G}$, or if Y or any I_i, O_i is the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y)] \parallel \overline{io}, ad)$
3. $c \leftarrow \texttt{challenge}([R], T)$
4. $\theta \leftarrow \top$ if $s \cdot I_m = R + c \cdot O_m$ else $\bot$

The identity is a legal value for R and MUST NOT be rejected. R is a prover-chosen commitment and step 4 is sound for any value of it. R is the identity only when $k = 0$, which makes $s = c \cdot x$ and publishes the secret key. A verifier that rejects it diverges from one that does not, on a proof that no honest prover produces.

3.3. Batch Verify

Multiple Thin VRF proofs can be verified together by combining the individual verification equations with random weights (Schwartz-Zippel lemma).

Input:

- For $j = 0, \ldots, N-1$: a tuple $(Y_j, \overline{io}_j, ad_j, \pi_j)$ where:
  - $Y_j \in \mathbf{G}$: Public key.
  - $\overline{io}_j \in (\mathbf{G} \times \mathbf{G})^{M_j}$: VRF input/output pairs.

- $ad_j \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi_j = (R_j, s_j) \in (\mathbf{G}, \mathbf{F})$: Thin VRF proof.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if all proofs verify, $\bot$ otherwise.

Steps:

1. For each proof j:
   (a) Validate $R_j \in \mathbf{G}$ and $Y_j, I_{j,i}, O_{j,i} \in \mathbf{G} \setminus \{\mathcal{O}\}$ for all i, output $\bot$ if any point is not in $\mathbf{G}$, or if Y_j or any $I_{j,i}, O_{j,i}$ is the identity.
   (b) $(T_j, (I_{m,j}, O_{m,j})) \leftarrow \texttt{vrf_transcript}(\texttt{ThinVrf}, [(G, Y_j)] \parallel \overline{io}_j, ad_j)$
   (c) $c_j \leftarrow \texttt{challenge}([R_j], T_j)$
2. Derive random weights:
   (a) $T_w \leftarrow \texttt{new_transcript}()$
   (b) $T_w.\texttt{absorb}(\texttt{BatchVerify})$
   (c) For each j: $T_w.\texttt{absorb}(\texttt{enc_scalar}(c_j) \parallel \texttt{enc_scalar}(s_j))$
3. Check the combined equation:

$$\sum_{j=0}^{N-1} w_j \cdot (s_j \cdot I_{m,j} - R_j - c_j \cdot O_{m,j}) = \mathcal{O}$$

where $w_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$.

Transcript:

$T_w = \texttt{suite_id} \parallel \texttt{BatchVerify}$
$\parallel \texttt{enc_scalar}(c_0) \parallel \texttt{enc_scalar}(s_0) \parallel \cdots \parallel \texttt{enc_scalar}(c_{N-1}) \parallel \texttt{enc_scalar}(s_{N-1})$

4. Pedersen VRF

Pedersen VRF resembles Tiny VRF but replaces the public key with a Pedersen commitment to the secret key, which makes this VRF useful in anonymized ring proofs.

The scheme proves that the output has been generated with a secret key associated with a blinded public key (instead of the public key). The blinded public key is a cryptographic commitment to the public key, and it can be unblinded to prove that the output of the VRF corresponds to the public key of the signer.

This specification mostly follows the design proposed by BCHSV23 [2] in section 4 with some details about blinding base point value and challenge generation procedure.

The *blinding base* $B \in \mathbf{G}$ is defined as:

$x = 23335687741101763108036518445642207119627658113885888016488710494487028845889$
$y = 55522145803750386930224096849798286003252109687457740808596604433373579299 63$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"pedersen-blinding"`.

4.1. Prove

Input:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

Output:

- $\pi = (\bar{Y}, R, O_k, s, s_b) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $b \in \mathbf{F}$: Blinding factor.

Steps:

1. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{PedersenVrf}, \overline{io}, ad)$
2. $b \leftarrow \texttt{blinding}(x, T.\texttt{fork}())$ (see Appendix A.4)
3. $\bar{Y} \leftarrow x \cdot G + b \cdot B$
4. $T.\texttt{absorb}(\texttt{enc_point}(\bar{Y}))$
5. $k \leftarrow \texttt{nonce}(x, T.\texttt{fork}()), \quad k_b \leftarrow \texttt{nonce}(b, T.\texttt{fork}())$
6. $R \leftarrow k \cdot G + k_b \cdot B$
7. $O_k \leftarrow k \cdot I_m$
8. $c \leftarrow \texttt{challenge}([R, O_k], T)$
9. $s \leftarrow k + c \cdot x, \quad s_b \leftarrow k_b + c \cdot b$
10. $\pi \leftarrow (\bar{Y}, R, O_k, s, s_b)$

4.2. Verify

Input:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (\bar{Y}, R, O_k, s, s_b) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

Steps:

1. Validate $R, O_k \in \mathbf{G}$ and $\bar{Y}, I_i, O_i \in \mathbf{G} \setminus \{\mathcal{O}\}$ for all i, output $\bot$ if any point is not in $\mathbf{G}$, or if $\bar{Y}$ or any I_i, O_i is the identity.
2. $(T, (I_m, O_m)) \leftarrow \texttt{vrf_transcript}(\texttt{PedersenVrf}, \overline{io}, ad)$
3. $T.\texttt{absorb}(\texttt{enc_point}(\bar{Y}))$
4. $c \leftarrow \texttt{challenge}([R, O_k], T)$
5. $\theta_0 \leftarrow \top$ if $O_k + c \cdot O_m = s \cdot I_m$ else $\bot$
6. $\theta_1 \leftarrow \top$ if $R + c \cdot \bar{Y} = s \cdot G + s_b \cdot B$ else $\bot$
7. $\theta = \theta_0 \wedge \theta_1$

Note: no public key appears in the verify inputs – verification uses the committed key $\bar{Y}$ from the proof.

The identity is a legal value for R and O_k and MUST NOT be rejected. Both are prover-chosen commitments and steps 5 and 6 are sound for any value of them. Both are the identity only when $k = 0$, which makes $s = c \cdot x$ and publishes the secret key. O_k is also the identity for every honest proof with $n = 0$, where $I_m = \mathcal{O}$ (see Appendix B). A verifier that rejects these values diverges from one that does not.

4.3. Unblinding

Links a Pedersen VRF proof to a specific public key. The prover reveals the blinding factor b, and the verifier checks that the proof is valid and that its committed key opens to the claimed public key.

Input:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- $\pi = (\bar{Y}, R, O_k, s, s_b) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $b \in \mathbf{F}$: Blinding factor revealed by the prover.
- $Y \in \mathbf{G}$: Claimed public key.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if the proof is attributable to Y, $\bot$ otherwise.

Steps:

1. Validate Y and $\bar{Y} \in \mathbf{G} \setminus \{\mathcal{O}\}$, output $\bot$ if either is invalid or the identity.
2. $\theta_0 \leftarrow Pedersen.verify(\overline{io}, ad, \pi)$ (section 4.2)

3. $\theta_1 \leftarrow \top$ if $\bar{Y} = Y + b \cdot B$ else $\bot$
4. $\theta \leftarrow \theta_0 \wedge \theta_1$

The verifier MUST NOT accept the association unless step 2 outputs $\top$ for the same $(\overline{io}, ad, \pi)$ that supplied $\bar{Y}$. Step 3 on its own is not evidence of authorship: any party can compute $\bar{Y} = Y + b \cdot B$ for a public key Y it does not control, without knowledge of the corresponding secret key.

4.4. Batch Verify

Multiple Pedersen VRF proofs can be verified together by combining the individual verification equations with random weights (Schwartz-Zippel lemma). Each proof contributes two equations (VRF correctness and Pedersen commitment correctness), each weighted by an independent random scalar.

Input:

- For $j = 0, \ldots, N-1$: a tuple $(\overline{io}_j, ad_j, \pi_j)$ where:
  - $\overline{io}_j \in (\mathbf{G} \times \mathbf{G})^{M_j}$: VRF input/output pairs.
  - $ad_j \in \mathbf{\Sigma}^*$: Additional data octet-string.
  - $\pi_j = (\bar{Y}_j, R_j, O_{k,j}, s_j, s_{b,j}) \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if all proofs verify, $\bot$ otherwise.

Steps:

1. For each proof j:
   (a) Validate $R_j, O_{k,j} \in \mathbf{G}$ and $\bar{Y}_j, I_{j,i}, O_{j,i} \in \mathbf{G} \setminus \{\mathcal{O}\}$ for all i, output $\bot$ if any point is not in $\mathbf{G}$, or if $\bar{Y}_j$ or any $I_{j,i}, O_{j,i}$ is the identity.
   (b) $(T_j, (I_{m,j}, O_{m,j})) \leftarrow$ `vrf_transcript`$(\texttt{PedersenVrf}, \overline{io}_j, ad_j)$
   (c) T_j.`absorb`(`enc_point`$(\bar{Y}_j)$)
   (d) $c_j \leftarrow$ `challenge`$([R_j, O_{k,j}], T_j)$
2. Derive random weights:
   (a) $T_w \leftarrow$ `new_transcript`()
   (b) T_w.`absorb`(`BatchVerify`)
   (c) For each j: T_w.`absorb`(`enc_scalar`$(c_j) \parallel$ `enc_scalar`$(s_j) \parallel$ `enc_scalar`$(s_{b,j})$)
3. Check the combined equations:

$$\sum_{j=0}^{N-1} t_j \cdot (O_{k,j} + c_j \cdot O_{m,j} - s_j \cdot I_{m,j}) + u_j \cdot (R_j + c_j \cdot \bar{Y}_j - s_j \cdot G - s_{b,j} \cdot B) = \mathcal{O}$$

where:

- $t_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$
- $u_j \leftarrow \texttt{dec_scalar_mod}(T_w.\texttt{squeeze}(\texttt{challenge_len}))$

Transcript:

$T_w = \texttt{suite_id} \parallel \texttt{BatchVerify}$
$\parallel \texttt{enc_scalar}(c_0) \parallel \texttt{enc_scalar}(s_0) \parallel \texttt{enc_scalar}(s_{b,0})$
$\parallel \cdots$
$\parallel \texttt{enc_scalar}(c_{N-1}) \parallel \texttt{enc_scalar}(s_{N-1}) \parallel \texttt{enc_scalar}(s_{b,N-1})$

5. Ring VRF

Anonymized ring VRF based on Pedersen VRF (section 4) and Ring Proof as proposed in BCHSV23 [2].

The ring proof can be seen as a special case of the Committee Key Scheme (CKS) introduced by CSSV22 [3], reduced to a single signer. In CKS, a prover commits to a set of public keys using a KZG polynomial commitment and produces a SNARK showing that a subset of keys – identified by a bitvector – belongs to the committed set. The ring proof is the degenerate case where the bitvector has exactly one bit set: it proves that a single (blinded) key is a member of the committed ring, without revealing which one.

The concrete specification of the ring proof scheme is given in VG24 [5]. The following configuration specializes it for this scheme:

- **Groups and Fields**:
  - $\mathbb{G}_{\not\kappa}$: BLS12-381 prime order subgroup.
  - $\mathbb{F}$: BLS12-381 scalar field.
  - J: Bandersnatch curve defined over $\mathbb{F}$.
- **Polynomial Commitment Scheme**
  - KZG with SRS derived from Zcash powers of tau ceremony.
- **Fiat-Shamir Transform**
  - `ark-transcript`.
  - Begin with empty transcript and "ring-proof" label.
  - Push R to the transcript after instancing.
- Accumulator base point $S \in \mathbf{G}$ is defined as:

  $x = 14056632001415368875257708737821299882600475929746323097150942355715730684350$
  $y = 10322661992765989500407719465917595459409463902187386706652408883505670839210$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"ring-accumulator"`.

- Padding point $\square \in \mathbf{G}$ is defined as:

  $x = 26913883415342152801331916189968962157924271221160514298872262294143390094043$
  $y = 30874728313203001508631936119690348239461579770372782660098261717479009115354$

A point with unknown discrete logarithm derived using the `hash_to_curve` function as described in Appendix A.2 with input the string: `"ring-padding"`.

- Polynomials domain ($\langle\omega\rangle = \mathbb{D}$) generator:
  $\omega = 49307615728544765012166121802278658070711169839041683575071795236746050763237$

- $|\mathbb{D}| = 2048$

The ring commitment and the ring proof depend on $|\mathbb{D}|$. Commitments and proofs made with another $|\mathbb{D}|$ differ, and they are not compatible with this configuration.

5.1. Prove

Input:

- $x \in \mathbf{F}$: Secret key.
- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.
- P: Ring prover (encapsulates ring keys and prover index).

Output:

- $\pi_p \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $\pi_r \in ((G_1)^4, (\mathbf{F})^7, G_1, \mathbf{F}, G_1, G_1)$: Ring proof.

Steps:

1. $(\pi_p, b) \leftarrow Pedersen.prove(x, \overline{io}, ad)$
2. $\pi_r \leftarrow Ring.prove(P, b)$

The blinding factor b is derived internally by Pedersen prove (section 4.1, step 2) and forwarded to the ring prover. *Ring.prove* and $Ring.verify$ are defined in VG24 [5].

5.2. Verify

Input:

- $\overline{io} \in (\mathbf{G} \times \mathbf{G})^n$: VRF input/output pairs.
- $ad \in \mathbf{\Sigma}^*$: Additional data octet-string.

- $\pi_p \in (\mathbf{G}, \mathbf{G}, \mathbf{G}, \mathbf{F}, \mathbf{F})$: Pedersen proof.
- $\pi_r \in ((G_1)^4, (\mathbf{F})^7, G_1, \mathbf{F}, G_1, G_1)$: Ring proof.
- $V \in (G_1)^3$: Ring verifier (pre-processed commitment).

Output:

- $\theta \in \{\top, \bot\}$: $\top$ if proof is valid, $\bot$ otherwise.

Steps:

1. $\theta_0 \leftarrow Pedersen.verify(\overline{io}, ad, \pi_p)$
2. $(\bar{Y}, R, O_k, s, s_b) \leftarrow \pi_p$
3. $\theta_1 \leftarrow Ring.verify(V, \pi_r, \bar{Y})$
4. $\theta \leftarrow \theta_0 \wedge \theta_1$

Note: to attribute a ring signature to a public key, apply the unblinding procedure of section 4.3 to π_p. The verifier MUST NOT accept that attribution unless this procedure outputs $\top$ for the same inputs, because $Ring.verify$ is what binds $\bar{Y}$ to a member of the ring. A $\top$ from $Pedersen.verify$ alone proves knowledge of an opening of $\bar{Y}$, not ring membership.

Appendix A. Concrete Instantiations

The following are concrete instantiations of the abstract interfaces defined in the main specification. They are provided to enable interoperable implementations and reproducible test vectors. Alternative constructions that satisfy the same security requirements are equally valid.

A.1. Transcript Construction

Instantiation of the transcript interface (section 1.6.1) using SHA-512.

Initialization: `new_transcript`() creates a fresh SHA-512 state and feeds `suite_id` into it.

Absorb: feeds raw bytes directly into the SHA-512 state. Consecutive absorb calls are equivalent to a single absorb of the concatenated data. This is safe because all protocol fields use fixed-width encoding (`enc_point`: 32 bytes, `enc_scalar`: 32 bytes, `enc_64`: 8 bytes, domain tags: 1 byte) or explicit length prefixing (ad via `enc_64`(`len`(ad)) $\|$ ad), so the byte stream is unambiguous given the inputs agreed upon by both parties.

Squeeze (counter-mode XOF): on the first squeeze call, finalize the SHA-512 state to obtain a 64-byte *seed*. Then produce output blocks:

$$block_i = \text{SHA-512}(seed \,\|\, \texttt{enc_64}(i)) \quad \text{for } i = 0, 1, 2, \ldots$$

where `enc_64`(n) encodes integer n as an 8-byte little-endian octet string. Each block yields 64 bytes. Output is read sequentially across blocks; partial block state is preserved between squeeze calls.

Fork: duplicates the full internal state (including any partial block position if squeezing has begun).

A.2. Hash to Curve

Instantiation of the `hash_to_curve` function (section 1.6.2) using the method defined in section 3 of RFC-9380 [6], with the *Elligator 2* map to curve (section 6.8.2) and `expand_message_xmd` with SHA-512 (section 5.3.1).

With one difference from section 5.3.1: the zero prefix Z_{pad} of `expand_message_xmd` has $L = 48$ octets, the expanded length of one field element $\lceil(\lceil\log_2(p)\rceil + 128)/8\rceil$, and not $s_{in_bytes} = 128$, the SHA-512 input block size. The expansion length is $2 \cdot L = 96$ octets. An implementation with the RFC prefix does not reproduce the constants of this document or the vectors of Appendix C.

This is the random oracle (`_RO_`) construction: the input is hashed to two independent field elements, each is mapped to a curve point via Elligator 2, and the results are added.

$$I \leftarrow \texttt{hash_to_curve_ell2}(DST, i)$$

The domain separation tag is:

$$DST = \texttt{suite_id} \parallel \texttt{HashToCurve}$$

i.e. the 27-byte `suite_id` string concatenated with the single `HashToCurve` tag byte (0x60). This matches the per-operation tagging used elsewhere in the protocol.

A.3. Secret Key Generation

Derives a secret scalar from a 32-byte seed.

Input:

- $seed \in \mathbf{\Sigma}^{32}$: seed octet-string.

Output:

- $x \in \mathbf{F}$: secret key scalar.

Steps:

1. $i \leftarrow 0$
2. $T \leftarrow$ `new_transcript`()

3. T.`absorb`($seed$)
4. If $i > 0$: T.`absorb`(i) where i is encoded as a single octet
5. $d \leftarrow$ `dec_scalar_mod`($seed$)
6. $x \leftarrow$ `nonce`(d, T)
7. If $x = 0$: increment i and go to step 2
8. Return x

The seed is absorbed into the transcript and also passed as a scalar to the `nonce` procedure (section 1.6.6), ensuring seed entropy flows through both the transcript state and the secret scalar input paths.

A.4. Blinding Factor Generation

Generates the Pedersen VRF blinding factor deterministically from the secret key and the VRF transcript state, using the nonce function (section 1.6.6) with a distinct domain separator.

Linkability warning: because b is derived deterministically from (x, T), two Pedersen VRF proofs with the same secret key, I/O pairs, and additional data will produce the same blinding factor b and therefore the same blinded public key $\bar{Y} = x \cdot G + b \cdot B$. An observer can detect that both proofs originate from the same signer by comparing $\bar{Y}$ values. Applications that require unlinkability across repeated proofs on the same inputs should generate b as a fresh uniformly random scalar rather than using this deterministic method.

Input:

- $x \in \mathbf{F}$: Secret scalar.
- T: Transcript state (from `vrf_transcript`).

Output:

- $b \in \mathbf{F}$: Blinding factor scalar.

Steps:

1. T.`absorb`(`PedersenBlinding`)
2. $b \leftarrow$ `nonce`(x, T)

Appendix B. Behavior with Zero I/O Pairs

When $n = 0$ no VRF output can be derived, since there are no output points to hash. The proof-of-knowledge component, however, remains sound in all schemes: a valid proof still requires knowledge of the secret key x.

- **Tiny VRF and Thin VRF**: Both schemes prepend the Schnorr pair (G, Y) to the I/O list before delinearization (sections 2.1 and 3.1, step 2), so the internal pair count is at least 1 regardless of the user-supplied n. With zero VRF pairs, the scheme degenerates to a Schnorr signature on the additional data ad, proving knowledge of x for public key Y.
- **Pedersen VRF**: No Schnorr pair is prepended, so with $n = 0$ the `delinearize` procedure (section 1.6.4) sets the merged pair to the identity: $(I_m, O_m) = (\mathcal{O}, \mathcal{O})$. The VRF output check $O_k + c \cdot O_m = s \cdot I_m$ degenerates to $O_k = \mathcal{O}$, which an honest prover satisfies because $O_k = k \cdot \mathcal{O}$. This is why section 4.2 step 1 accepts the identity for O_k. The commitment check $R + c \cdot \bar{Y} = s \cdot G + s_b \cdot B$ still proves knowledge of the Pedersen commitment opening (x, b).

Such a proof carries no VRF output. An application that consumes a VRF output MUST require $n \geq 1$ at the call site. The schemes do not enforce it.

Appendix C. Test Vectors

The test vectors in this section were generated using `ark-vrf` version `0.6.1`.

C.1. Tiny VRF Test Vectors

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
proof_c: Proof 'c' component,
proof_s: Proof 's' component,
```

bandersnatch_sha-512_ell2_tiny - vector-1

```
c9922b7a9849b9928e15c655dd2f22ceef737cc355024f43d4b04bf4398c270d,
5a538209ff1fc7b1c9c8e1da05b3e169acf10a8b1591b3af029fe4eede0bbc71,
-,
-,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
54421f7ffc399872f1cb868efbb7eef4034178f1e369cebfb964ca61e4f3f256,
7b89f2aba6af7474694f24f75adf48336e00dcc8f3ac889ef4daa53c859497a6,
5685489f948058d1ac34ffc87d6adc09,
be881519a8790c307e3997b9b7905061b82009bc9d73ee5c5363319b1c884915,
```

bandersnatch_sha-512_ell2_tiny - vector-2

0b4259ca1b10c9ed462532639113e1caf26b3a1a2d9e91ecef2fc5c2d23aed0a,
ff341f0c9da793b2d8fef91bcbfd5b55c2185352e4289edc1dd6c64e3fe09b0d,
0a,
-,
8ec04d55a790d47cd32c5062cb44517f164515dac88a8ea6d972db1a7da08abf,
e20e48a14d1ab23cad04e8bc39705d194e87587072fe2e7114d1f85fd7fb6105,
f7f5da876f037509854307d89273b14ef5db4f0971df0879963e1bf0e775636f,
8d56976c451b512a88a28704cd62546e,
68a6a31d6ce21ac9ea4e6eb688e19e08fc30a50a0dcd7c459dc0be49daaad415,

bandersnatch_sha-512_ell2_tiny - vector-3

dd60163595ff312a49aa5849917ba19020038ccd42f8a1d468da0973079bdb15,
fbba8feb488e767b9864726fffdc8595896757430eba9162a1a5d9a03381d5a4,
-,
0b8c,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
719ace6342e2c5b3615e33d6e081c6ffbf0813f3f42ccbdf92973869fab0bbe5,
cc87c37551a222fe1207655f20225cb79a3dc2ebcefd7111550911e36423ad62,
7d7a08eed618b81bef1ca85695b93d6d,
3a1107ff4361fd10bf5b891d5554cc76e56502d9f034e921eb2cac0aa73d9206,

bandersnatch_sha-512_ell2_tiny - vector-4

dfc32f03fe9487f123f2afeeb9487cb6b1eb23efac24a60ae540f5aa632ddd18,
f8487052801a89161424ee745189b5f7fe568819b9f13f44a8b3d173b57e928a,
73616d706c65,
-,
fa6a45172b622ddaccbcf7a3b2d91a36ba504598a83d5a4ed23b416063e4e039,
8f3ee5f3e386775c13093f84678e01594419d747a7037619fa7ea20790044218,
67c6fbdf10f3b5abec5c5ea368662455aa1cac36a6a17621ddb2b526c52fd8dc,
e44e5064895730cb04907d46262e63fa,
393f12e07c2ce66c08855bcf64a9c3b1dc29e455af7e235500f04076edb0c70e,

bandersnatch_sha-512_ell2_tiny - vector-5

04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
-,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
c43dfc9ad3e72ab427810f5879683725,
08484f31d398c8f27b8ff363354bb14a5b5c23caf1436b321f5ea30b7f555e13,

bandersnatch_sha-512_ell2_tiny - vector-6

```
04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
53613c4e23191e4bc2632927f7fd4316,
84e4072747518ad201335c841fac19222f8ee895f288da6304cd89b3ab4c7708,
```

bandersnatch_sha-512_ell2_tiny - vector-7

```
9504efbeadf81b20a9cb64c1331915eb3718a574227458230d1d80dfa94e8b13,
704fd3784947de4db4fdcf0b477530d094bf5a656707b7d6cb43edbc4db7336b,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
1617b4fca3d7354fd43ae783bd6d0cfc526a80e257fcccfdf8e4d87e74dd7c6d,
6b279b85a1c55f55db80167e6bee997a5bf25c1767ffc2a34ae696baf606f5a0,
365b235ddb44510f484f14d5d19a1cd6,
9f5f9515507c84924f87b5cf549da8b177879cd2f835f94b91e36543e0cfbb0c,
```

C.2. Thin VRF Test Vectors

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
proof_r: Proof 'r' component,
proof_s: Proof 's' component,
```

bandersnatch_sha-512_ell2_thin - vector-1

```
c9922b7a9849b9928e15c655dd2f22ceef737cc355024f43d4b04bf4398c270d,
5a538209ff1fc7b1c9c8e1da05b3e169acf10a8b1591b3af029fe4eede0bbc71,
-,
-,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
54421f7ffc399872f1cb868efbb7eef4034178f1e369cebfb964ca61e4f3f256,
7b89f2aba6af7474694f24f75adf48336e00dcc8f3ac889ef4daa53c859497a6,
d969c67f8fd97fcc8c0f8b129810ef5427786e319e304a056038caf32286d15e,
374e93cc0df1aaa3cbe7a66397213275e852717462f8a9b775ed8bcb68f45b00,
```

bandersnatch_sha-512_ell2_thin - vector-2

```
0b4259ca1b10c9ed462532639113e1caf26b3a1a2d9e91ecef2fc5c2d23aed0a,
ff341f0c9da793b2d8fef91bcbfd5b55c2185352e4289edc1dd6c64e3fe09b0d,
0a,
-,
8ec04d55a790d47cd32c5062cb44517f164515dac88a8ea6d972db1a7da08abf,
e20e48a14d1ab23cad04e8bc39705d194e87587072fe2e7114d1f85fd7fb6105,
f7f5da876f037509854307d89273b14ef5db4f0971df0879963e1bf0e775636f,
22be463a480dfadb3beb93ba75890a612fb65e206fecdd9fa9ec130ed16db268,
2eb15dc87c37022290606c34db23fdd7a80648e6aa0dce076ca3694000a72c01,
```

bandersnatch_sha-512_ell2_thin - vector-3

```
dd60163595ff312a49aa5849917ba19020038ccd42f8a1d468da0973079bdb15,
fbba8feb488e767b9864726fffdc8595896757430eba9162a1a5d9a03381d5a4,
-,
0b8c,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
719ace6342e2c5b3615e33d6e081c6ffbf0813f3f42ccbdf92973869fab0bbe5,
cc87c37551a222fe1207655f20225cb79a3dc2ebcefd7111550911e36423ad62,
2d726da5bc791c511e7b5629bde266151d66f9f36d276dbe16dc1c900f4913dd,
743aaadff73eaaccc8e6cc333542faa331a132012e9cb2112a1e681546152d01,
```

bandersnatch_sha-512_ell2_thin - vector-4

```
dfc32f03fe9487f123f2afeeb9487cb6b1eb23efac24a60ae540f5aa632ddd18,
f8487052801a89161424ee745189b5f7fe568819b9f13f44a8b3d173b57e928a,
73616d706c65,
-,
fa6a45172b622ddaccbcf7a3b2d91a36ba504598a83d5a4ed23b416063e4e039,
8f3ee5f3e386775c13093f84678e01594419d747a7037619fa7ea20790044218,
67c6fbdf10f3b5abec5c5ea368662455aa1cac36a6a17621ddb2b526c52fd8dc,
9a638cee379161ecf1a9269801631303359a531b035076369617c42ef46afcb1,
223446d4fd1450c1fef095c3fd3d5a52ae9f3a8b07a88d7e7ddaed089b235504,
```

bandersnatch_sha-512_ell2_thin - vector-5

```
04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
-,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
fdfae23d8d8c9a5225a7bd49b65cc276abf7ce0f48a3730ca74eb57fe0b27467,
d55860b9c11a56cbe92b5d9c992fd95756fe1845be0be7eaa2a291b524cd111a,
```

bandersnatch_sha-512_ell2_thin - vector-6

```
04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
d415fcd6eff582440fab9c62819b3bb82dd7158a2faf0465512aa095e9e885c2,
f2f59942f122f9566576ff9cfffbc803f1229881044aa69cca96193106b08c0c,
```

bandersnatch_sha-512_ell2_thin - vector-7

```
9504efbeadf81b20a9cb64c1331915eb3718a574227458230d1d80dfa94e8b13,
704fd3784947de4db4fdcf0b477530d094bf5a656707b7d6cb43edbc4db7336b,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
1617b4fca3d7354fd43ae783bd6d0cfc526a80e257fcccfdf8e4d87e74dd7c6d,
6b279b85a1c55f55db80167e6bee997a5bf25c1767ffc2a34ae696baf606f5a0,
9f9f6670d06734e319a01c6afb72e0e3647085dae2216c707a22643a3ef1a5c2,
74544f931a955eb7f38dab224988f37425b1275b78ed4340f6d5b7c4bc91a60a,
```

C.3. Pedersen VRF Test Vectors

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
blinding: Blinding factor,
proof_pk_com (Y^-): Public key commitment,
proof_r: Proof 'R' component,
proof_ok: Proof 'O_k' component,
proof_s: Proof 's' component,
proof_sb: Proof 's_b' component
```

bandersnatch_sha-512_ell2_pedersen - vector-1

```
c9922b7a9849b9928e15c655dd2f22ceef737cc355024f43d4b04bf4398c270d,
5a538209ff1fc7b1c9c8e1da05b3e169acf10a8b1591b3af029fe4eede0bbc71,
-,
```

-,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
54421f7ffc399872f1cb868efbb7eef4034178f1e369cebfb964ca61e4f3f256,
7b89f2aba6af7474694f24f75adf48336e00dcc8f3ac889ef4daa53c859497a6,
afa1000852c300cbf235a6955ec4105b01919fa67ec6cc92ec92100f19c25809,
680539c070ebcaca50c4bf201869c2c355f1467e1b8ece6e0bcafc466872c5cf,
375a8ec2e95f58a41ea09090f9560e580e7a3cc67998129fa98dac1da5d0850e,
5071efbef9428b2a26eb09440891c6829bc0c57b4254ca811ce453a0d5938dea,
7dc8aa043b28383d334ef500fd7d770d35ab1c5ebc28df0807215911f4ab2d10,
045343af67f88d35679beab125c38b80029099b746ea47ef3eb488b1d38d130d,

bandersnatch_sha-512_ell2_pedersen - vector-2

0b4259ca1b10c9ed462532639113e1caf26b3a1a2d9e91ecef2fc5c2d23aed0a,
ff341f0c9da793b2d8fef91bcbfd5b55c2185352e4289edc1dd6c64e3fe09b0d,
0a,
-,
8ec04d55a790d47cd32c5062cb44517f164515dac88a8ea6d972db1a7da08abf,
e20e48a14d1ab23cad04e8bc39705d194e87587072fe2e7114d1f85fd7fb6105,
f7f5da876f037509854307d89273b14ef5db4f0971df0879963e1bf0e775636f,
da3304d52c2a64ea75504ac44776e5e4900a38eb9a61f9478285aeccba6ec41c,
a356a2fafe785bc79860ef65534df7c1a2ee7d45ff6db649537dac1687ae4785,
ec546c1932f72e146a0e45d7107c97e99aacc9c72fd6ae6fd0b0417124aa26ca,
1d661c6349b33182ec77b58457f6d45dd9a618017ea021b986cf1eff18c4ef95,
1e8eb9a8cf017c6a50d0f947f9bf02575b60c714c81602a8f06965ec7e67c300,
49c44ad19ea4aaadfc98c2586660dbf3d8a434bb062fa948d82fd4a717ac5f12,

bandersnatch_sha-512_ell2_pedersen - vector-3

dd60163595ff312a49aa5849917ba19020038ccd42f8a1d468da0973079bdb15,
fbba8feb488e767b9864726fffdc8595896757430eba9162a1a5d9a03381d5a4,
-,
0b8c,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
719ace6342e2c5b3615e33d6e081c6ffbf0813f3f42ccbdf92973869fab0bbe5,
cc87c37551a222fe1207655f20225cb79a3dc2ebcefd7111550911e36423ad62,
3dd321bf1fbbafd76d48c000a28768152e919789bd57cd50d5e17c88a1baec0a,
6d16e2ff7b28a8880499e34cfe6ffbb1d12efc27675af3fef635e8761c893dee,
5311bd0755a2bbc6db01b930526e85120af5189e2fb259fc941f3acb1611d6eb,
3c4ea965693b470b0da4f9c9508da97dfa70e5e85ee78fbd3af377a2298b4864,
a48a2f80da4190ec19e293a50bbc1bc047ebc38a14f4f19f97f5daec48ab4205,
eb91c85fb3b211f1ec63612950d04c78fc19a760294d01bd82f9218e944b640d,

bandersnatch_sha-512_ell2_pedersen - vector-4

dfc32f03fe9487f123f2afeeb9487cb6b1eb23efac24a60ae540f5aa632ddd18,
f8487052801a89161424ee745189b5f7fe568819b9f13f44a8b3d173b57e928a,

```
73616d706c65,
-,
fa6a45172b622ddaccbcf7a3b2d91a36ba504598a83d5a4ed23b416063e4e039,
8f3ee5f3e386775c13093f84678e01594419d747a7037619fa7ea20790044218,
67c6fbdf10f3b5abec5c5ea368662455aa1cac36a6a17621ddb2b526c52fd8dc,
b3f08962012e4f4aa9e6482d7d682275ac98e57f099b991947fad5a53451d51b,
1c48e5ce0ae9b3e7a09626bc1333544f1dbe5033d5efd591a059b04a41830eb6,
17b630103915e544c1b4b1a3e8cbad9cf57e75bf7f02a18ae965db26f0d2189d,
a16224226f643245a4c42ddac28d614fbe46d02ea1f8d1dcab91a26eea0c19bb,
bd55995a3f6bffe519366ea7b891e1d006da5472b8de95c5a8c7db52adf2f91a,
c946c76414f1c4b5f5825e4cb5fcda7a5aacde9fa0ce1c478caec1f3d812c606,
```

bandersnatch_sha-512_ell2_pedersen - vector-5

```
04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
-,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
08fbea5cbe5af93a1529c222d6b3fcc672599e59261d9ed485ef45146befeb03,
cdf7907e1902ab6989732bd58241719129057252 96e86d2d864ed0eea5322d54,
02667d1f512990e80b127eed9139ae5590853fde2469e177fee5fa0908de1fc2,
c2d48dada6adf017c2ed658fd91789e4b63f2d6b894ce98fbfef97c56ed4d5d5,
48c13fd4d2cbe731e7ecbf9cef6b2fa5c1a55f27bc0f34753cc211d7cbff7103,
78c5b906fbbca6466120cd079bf7fcdecedc1db7916ed74dbf3ddc2f5063b81a,
```

bandersnatch_sha-512_ell2_pedersen - vector-6

```
04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e617463682076656374 6f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
c8b1ce7a665f75506e6a64ef52aa7d8cbfecde0ad83946eb980ba4d06580f819,
635ccf83a424afe87946a36a532f2f2dd4e72b280f9a8d04e4abe3b7f3cbd89c,
3b15fefece933682d3a92dc57fdaa4da6e76f80c41cb926b00376576bced3668,
4b24cd4a4c25aa29aae4e969cf5be562cb9ea176593f8c06bb7343c8da71eac0,
3278c30b9d2fd950fa1a6e82e98375c8fd651f237806f8eaab44c0817b2cad10,
d59f47aa43f471c63ff9cebbab2aaea14feeeb914a3e1a9589350864f3d7b902,
```

bandersnatch_sha-512_ell2_pedersen - vector-7

```
9504efbeadf81b20a9cb64c1331915eb3718a574227458230d1d80dfa94e8b13,
```

```
704fd3784947de4db4fdcf0b477530d094bf5a656707b7d6cb43edbc4db7336b,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
1617b4fca3d7354fd43ae783bd6d0cfc526a80e257fcccfdf8e4d87e74dd7c6d,
6b279b85a1c55f55db80167e6bee997a5bf25c1767ffc2a34ae696baf606f5a0,
3a19e858c8f2cc41037737c43d23dbf3d124ea7059d30c6ab88aa184e14fe20c,
0a9e31ccc09e5464e53afa2057098da6208478f9334199c60bd4bc7443a3d5c5,
1ce7b8f6bcf3723106588842207b1199b8feaaf3bcc13874c0fa2e82a53b47b3,
7372c8e03008802e1bfdf4ac6b8832aded5b48ddb688af168cb67a882628ba22,
4b61c4d7d6179d8e1fe407a20e1ab22acec0a4834eb36fe03f254480d9d87e1b,
40d1a91ac8b287a2b754e9ba05c7c2091e140f0fb40753ebbd2573b6e9d1f514,
```

C.4. Ring VRF Test Vectors

KZG SRS parameters are derived from Zcash BLS12-381 powers of tau ceremony.

The ring has 8 public keys. The vectors use the domain of section 5, $|\mathbb{D}| = 2048$, with the generator ω of section 5. This domain holds 1791 keys. The padding point $\square$ fills the key positions after the 8 ring keys.

The ring commitment and the ring proof depend on $|\mathbb{D}|$. With the smallest domain for 8 keys, $|\mathbb{D}| = 512$, an implementation does not reproduce `ring_pks_com` and `ring_proof`.

The evaluations for the ZK domain items, specifically the evaluations of the last three items in the evaluation domain $\mathbb{D}$, are set to 0 rather than being randomly generated.

Schema:

```
sk (x): Secret key,
pk (Y): Public key,
in (alpha): Input octet-string,
ad: Additional data octet-string,
h (I): VRF input point,
gamma (O): VRF output point,
out (beta): VRF output octet string,
blinding: Blinding factor,
proof_pk_com (Y^-): Pedersen proof public key commitment,
proof_r: Pedersen proof 'R' component,
proof_ok: Pedersen proof 'O_k' component,
proof_s: Pedersen proof 's' component,
proof_sb: Pedersen proof 's_b' component,
ring_pks: Ring public keys,
ring_pks_com: Ring public keys commitment,
ring_proof: Ring proof
```

bandersnatch_sha-512_ell2_ring - vector-1

```
c9922b7a9849b9928e15c655dd2f22ceef737cc355024f43d4b04bf4398c270d,
5a538209ff1fc7b1c9c8e1da05b3e169acf10a8b1591b3af029fe4eede0bbc71,
-,
-,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
54421f7ffc399872f1cb868efbb7eef4034178f1e369cebfb964ca61e4f3f256,
7b89f2aba6af7474694f24f75adf48336e00dcc8f3ac889ef4daa53c859497a6,
afa1000852c300cbf235a6955ec4105b01919fa67ec6cc92ec92100f19c25809,
680539c070ebcaca50c4bf201869c2c355f1467e1b8ece6e0bcafc466872c5cf,
375a8ec2e95f58a41ea09090f9560e580e7a3cc67998129fa98dac1da5d0850e,
5071efbef9428b2a26eb09440891c6829bc0c57b4254ca811ce453a0d5938dea,
7dc8aa043b28383d334ef500fd7d770d35ab1c5ebc28df0807215911f4ab2d10,
045343af67f88d35679beab125c38b80029099b746ea47ef3eb488b1d38d130d,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..5a538209ff1fc7b1c9c8e1da05b3e169acf10a8b1591b3af029fe4eede0bbc71
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
89ed14c3a1a49a231e869c45b1ad28868847f850a8ba9281507e7a875bcde2c6
..de4e91be9469e80c77fbce0edfb1af1b92ff03f0316afef177a5cd762dab0e29
..1e6117b40c755adaca34b4fa0dd7394db2f345273aaff48e0e66b09a0de66019
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
af2328592c150be3c8370ef26d70faace8d8609a797db7198f0b7932776e70f7
..6e9c87d2e8339d30c864fe473235691fb4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..b0196940ff389d968f17fe91c56eb57c42f88a95c35a9d32a3c1934195d2c489
..42a46eaf84b231095544e1f3c244f509ad4bb837c7f6e71f2b7ed2915f374dcc
..6ed49175cb88c7ce88062e89f1600010c72e54dd876ecc241f28a28d9127031b
..1bb2fe685725ccba2bed37438d71b7049ac1a2204897869ddce2452d9a92491d
..40481d321a046f00a6b34bfabe564435f889d7eeafc485b28e6b238a4a102c2c
..c76442b496ebd63ba1a5abee95d6303229e148f7b31303664da7c62db27e9e32
..04c70e886f3ce8cf35f378bb59b80ef21ebb69b944050846801e061b8325e247
..482505666e4733568084367262c0df2d557008c548ca6538376c0306bf89e571
..9589944156d4172455d5aecd61fe6c1953448d18a3a09de861026213f970476b
..82b67719e323581f1ac81c308331ee1d8cb89cdd34ca3a8b7bd96445b31e2727
..8fdb6913c7189209a043fc61af9ea3cbabcd5fc9add8b787e175913b95c5a7a0
..d37efc37570652d79ee8af0f591bdc755eb29dccdd4e861705bd55ab09f6e9de
..415da4fcffd422d41963002323e159049320275f51dac5835544d6e7f6a78461
..532835cf1eacb49030463ec3b574139b8be701d041fb13a26b49f64ce96ffc43
..8cc71e8bca624ce470d445fb26cd1162873fc420399a48610b1c37320a9f4d4e
```

```
..fe2a20d0129035154ac58985458eb865,
```

bandersnatch_sha-512_ell2_ring - vector-2

```
0b4259ca1b10c9ed462532639113e1caf26b3a1a2d9e91ecef2fc5c2d23aed0a,
ff341f0c9da793b2d8fef91bcbfd5b55c2185352e4289edc1dd6c64e3fe09b0d,
0a,
-,
8ec04d55a790d47cd32c5062cb44517f164515dac88a8ea6d972db1a7da08abf,
e20e48a14d1ab23cad04e8bc39705d194e87587072fe2e7114d1f85fd7fb6105,
f7f5da876f037509854307d89273b14ef5db4f0971df0879963e1bf0e775636f,
da3304d52c2a64ea75504ac44776e5e4900a38eb9a61f9478285aeccba6ec41c,
a356a2fafe785bc79860ef65534df7c1a2ee7d45ff6db649537dac1687ae4785,
ec546c1932f72e146a0e45d7107c97e99aacc9c72fd6ae6fd0b0417124aa26ca,
1d661c6349b33182ec77b58457f6d45dd9a618017ea021b986cf1eff18c4ef95,
1e8eb9a8cf017c6a50d0f947f9bf02575b60c714c81602a8f06965ec7e67c300,
49c44ad19ea4aaadfc98c2586660dbf3d8a434bb062fa948d82fd4a717ac5f12,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..ff341f0c9da793b2d8fef91bcbfd5b55c2185352e4289edc1dd6c64e3fe09b0d
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
8601b512bffff9c078c916757752200cffd9d1c247b21991076c0475a3406b2e
..8e9dd85859de69b4db70915f8da8fa2a95f25aacbd9647719de0bcab840f2ab8
..ac1c79f55c920e8a3b678ccde2876ff1319107bf160940069576cbc6d1fc3d02
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
90d05a34161bf52f9b133ad8321b59cc80e3df751a736c5820e0d9c00760cd4d
..0ce8ea76e29b9e556c57810b8f761d89b4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..b339c2c78d1a4a4bd1469fcf36779dc92b07e3e2778b1281105a160b6ed42b04
..af95c875e5e753ea7a3719c114d0e9179040502c5a7456f257a0a05813341534
..88c57aadf4358f759140ee1d09ffa16018b3b638f16bf3afd2d4101eb814e4e7
..12fbb9414663aefc32a8b89e8b23d9df3c763ef774fe0f31cf8647264c9e4a17
..b3c7de31e04fea986b000532dde7f7f83d276f23b8218daa1cc7231a6ed8bf3a
..43dd740818abfb79e291deea093577bf341f1625ac7376dac23d1bde4d26d948
..04856690719b3a34606f44221705bda0260ca7ef3ef89c5412ae890c131efc37
..d1737d4288382fdd52c58a045f64458a1ef5b2770a7988945d012b7239caeb54
..8f78d73f7e9a4cd47ecf58f14a0d24bb8806ea96d42d8ee308d0f0bc2578041e
..50e6d4ca693693009c3b7077ed622b226ad5da61e2a3febd10d558d9dbeab30a
..909bc6f758e0c0401b142308fec51b43e0c8c3b8c7543eaf0454081a6435c151
..3812762b7fbfd5436dc06e8b958410cbe38ef37b63ce21bd5ac41ec026426b3d
..c8ac7030a8a4c07da2eb7b0e497f1c048113369742a3965e05898250847efd5e
```

..92bdccccfb785219ff3aa96a1eaa5869f8d0172640e0bc7afbb305131294ab55
..b200c5faba2a29af7c7181a6e43016269af048d12f68539c71daf97f111ab01f
..2b8b97cc3ce423a7d915827c3321d54f,

bandersnatch_sha-512_ell2_ring - vector-3

dd60163595ff312a49aa5849917ba19020038ccd42f8a1d468da0973079bdb15,
fbba8feb488e767b9864726fffdc8595896757430eba9162a1a5d9a03381d5a4,
-,
0b8c,
f508a4e84812ee3dce73ef72bb9064308128384b4801f81ef8616a7dffc486bc,
719ace6342e2c5b3615e33d6e081c6ffbf0813f3f42ccbdf92973869fab0bbe5,
cc87c37551a222fe1207655f20225cb79a3dc2ebcefd7111550911e36423ad62,
3dd321bf1fbbafd76d48c000a28768152e919789bd57cd50d5e17c88a1baec0a,
6d16e2ff7b28a8880499e34cfe6ffbb1d12efc27675af3fef635e8761c893dee,
5311bd0755a2bbc6db01b930526e85120af5189e2fb259fc941f3acb1611d6eb,
3c4ea965693b470b0da4f9c9508da97dfa70e5e85ee78fbd3af377a2298b4864,
a48a2f80da4190ec19e293a50bbc1bc047ebc38a14f4f19f97f5daec48ab4205,
eb91c85fb3b211f1ec63612950d04c78fc19a760294d01bd82f9218e944b640d,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..fbba8feb488e767b9864726fffdc8595896757430eba9162a1a5d9a03381d5a4
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
8cd303758e5971d9898227b02307acdd0f904f688e688a65f5af1f7319b49430
..6e07576d35a4695c23dd8c1c480b2fe7940ffecf9111dd55f46c6d864b4c0f16
..e07473cff1157417b7d87b6d1929942da23965b474d2cef8bf7cf48b5dee9a78
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
add55da9ea1146a058c7fd21032fc33c86e67731119c2090ce59310f555fd088
..eab1cd8b4803868d1b2938a2b2ad017bb4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..b79e3ab9b143f7b2f9a100554cf67bd8c4ab433454a4a3e980678351fd20bd58
..f3be8c361ffbfba3a8ff98ce076f6e4da7e74a9fc31072fa5a3b5063c549214b
..a65a1613b186447b0ab6e708296de37b5ff5a44d47fb51d3eba20c40b17823b4
..31257642a1bde16bb301e05a8dcc1d7b486d93644ca286d5cd83a1c6b4313656
..3ac7aee7859836812adeca4101789cfe8c02748f860e8993651f46cc2a03fc06
..3193048e62c5235f15503ab7e6296396ff31081ab5312be487655c140cf32417
..31261714f11d6490a37c4994a28d3424f36df28447e458e588d25c317f99b134
..cae75561d51dd83109b607c24ceec5f38523a936618fed12852f01f5a1075c5e
..c0c9ce2f0174dd30961b874a5b2e9e0b583c0037def361da338b20358fdbef30
..7393a7e2f04a27165d1b85d375036b3ec446676ea838cd4baeaf1f2aa225dc08
..b7e4849d032bbc8f25cd84d0ff9cfcc4ebd94bdb287ad9f196af6fd3b912eb7f

```
..7e7bc1d9cb68393069173c8e827469f59d22128e254b88288a5424b534bc356f
..616371a730e42775d10730806feeb06fa17f53645bdf5a8d3572362cf5c27c1f
..521b8d611c369381d4101efcf941ad91a1bd1666951ac6aac881f56cebdae17e
..b18f3b12ab2c76255fe7516b349a19f0eac21ceda7fcbaabf84a3011727a6f31
..ddb7f5d93c7ad3a8456939cfee5df3f0,
```

bandersnatch_sha-512_ell2_ring - vector-4

```
dfc32f03fe9487f123f2afeeb9487cb6b1eb23efac24a60ae540f5aa632ddd18,
f8487052801a89161424ee745189b5f7fe568819b9f13f44a8b3d173b57e928a,
73616d706c65,
-,
fa6a45172b622ddaccbcf7a3b2d91a36ba504598a83d5a4ed23b416063e4e039,
8f3ee5f3e386775c13093f84678e01594419d747a7037619fa7ea20790044218,
67c6fbdf10f3b5abec5c5ea368662455aa1cac36a6a17621ddb2b526c52fd8dc,
b3f08962012e4f4aa9e6482d7d682275ac98e57f099b991947fad5a53451d51b,
1c48e5ce0ae9b3e7a09626bc1333544f1dbe5033d5efd591a059b04a41830eb6,
17b630103915e544c1b4b1a3e8cbad9cf57e75bf7f02a18ae965db26f0d2189d,
a16224226f643245a4c42ddac28d614fbe46d02ea1f8d1dcab91a26eea0c19bb,
bd55995a3f6bffe519366ea7b891e1d006da5472b8de95c5a8c7db52adf2f91a,
c946c76414f1c4b5f5825e4cb5fcda7a5aacde9fa0ce1c478caec1f3d812c606,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..f8487052801a89161424ee745189b5f7fe568819b9f13f44a8b3d173b57e928a
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
abac1a85a16139a272f0a9bd44d16d1a35958a67ee2a3aaee25a23fc6287827e
..43a2e05cfc23fe4fc4b8ff97783510f197127760b747e687f5affeca955f3116
..8dbccc55188a16b9fa7cdd3c55f22fc6cf28024ced23e184d8f108e8d5461449
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
83b9765fc2040f9dc45a150b91d087c97c51d4001e1ef202ce559746aaa715ac
..4813b3e61ebdf636f427aa50a273eb13b4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..b903082c4d49eb7df505e33d913eec61daf4ba473e212ce0e358fc8c9d0a7e29
..c109f3a65e403181d79423065e5689be81296989656548b8a4905c7d3c97460e
..26dc66f527f14bd5773ab2688c356c8b6d6fe973af8598401553d3a873c855fb
..de586b0baba617bd5c7340631e369ce9cf0af794cc44edd383c7a64bd5c79343
..2faad03c5284be1888008cde08965d40b380bc456962c0d1fa28b01a2b667660
..d02a99c9d39c17da02acb76ec993c528b413b6260ba344889ea6e176b483381a
..c2764e6126c0331a1fc74fca2082623a2ebba161854c732ffca6553fbba11854
..72a27a933ca7df3688945609ba86ff5bf9ce0b3f7fd3a07aecc290f0b7f9d071
..2a4cb789bffe639fdc2e9a5322bb32097805d1792d7766d35fcc2b26d12b2910
```

..811e1c7739dca8fd57717d4b31eb093d37208fa7f0f39d862dda722fcb3ff01d
..b906ed1e54e7af58ba1e0c63d8378f376a12fa1c77e0899c0b9a70f6f9d65e88
..ae5beb696ed90c7f8bd52e6f33d9334576867331ccb8d473f05e53833276e5db
..edccbcbd5a0969ecc1bb794e6d3ad82ab6e71333cbaf0e88e623cf905b55417c
..dc8f9d24af8396cedf2cf1d624f5a63105eec4e81cabb80622db225904f1248f
..9899c3b46611e66db14531234347116d5177d5f9d09845255ffd65a429d88c1b
..325a235b43dab33ccd2311c3b50e94c6,

bandersnatch_sha-512_ell2_ring - vector-5

04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
-,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
08fbea5cbe5af93a1529c222d6b3fcc672599e59261d9ed485ef45146befeb03,
cdf7907e1902ab6989732bd5824171912905725296e86d2d864ed0eea5322d54,
02667d1f512990e80b127eed9139ae5590853fde2469e177fee5fa0908de1fc2,
c2d48dada6adf017c2ed658fd91789e4b63f2d6b894ce98fbfef97c56ed4d5d5,
48c13fd4d2cbe731e7ecbf9cef6b2fa5c1a55f27bc0f34753cc211d7cbff7103,
78c5b906fbbca6466120cd079bf7fcdecedc1db7916ed74dbf3ddc2f5063b81a,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
a3d604cc19e3ce06e7308730afb0637dc91f36160df749b86d9a8f21fd4608ba
..85c445bcba1951109771dcd3985f3e6ca88d087424d9dcb55ecce7ee85a2814b
..0263d0b9b22523c19efe00cf7038ccd4f9ba46212ca811dd272fbc0862c9fc92
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
a4860dc5c8355dad7f91a20f10db20ff8fb9ee73ee83560ab6bca438a0099a51
..ce529af30dded6959a297c06ea191623b4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..a51e0634529ec30e449719299db31528356a45e646546c3d851f1e97bd4408c7
..0f8f71adefa466e047206c41edc71a7a8ae0f53a6874b3c8c0d09bf80c868267
..4ab19f88fd6cff8e242f4fe05b35aa6ee77fae55bbf19ee8069d475aa6960d19
..1949554571313a96590230dff12e03fa9676e7ee5a3f810f757902e6686b3825
..b38e5c20910dc9967bff9b1d5d8844b6d5346533fa280e4e05eaeef18b64f218
..0a9bc9a9bea499251f84a3f54f4f639718f6f2cd545a13226df7cedc6940db24
..4b0daad111bc8094c1fdf2396dd18397d8f26cac1d9a3e1613455b8c92ea2a6e

..d730a99e0ab7ddf28226963d4dbd218c06d958ee0f1086992bb89ad05cd05d6d
..bb2843419d8785df035e45a8a1908ce69cd0e62548e9e0c32a6e54ad0718f839
..071a1d502e3187cb91e7bb2c32820f751baf0e829c9f0d58fcac3d68fef13a2b
..80dc68e7f63a91d26edfedc47683bd64bbb57f6e74e21e9b7cccf31c42d481b7
..778d73aa719b128d0b62447a394876262ed5dee56b4bd0a4f59535cd74bfbb76
..7c07a7e59eaa3b02b58dbc9b73f5d65fa816d0a47fb7283d8f77e4d375cedd09
..13d1b6e528f33c70cf49a0e186aff883fc3a23a335dedfdef7194df33a2b3918
..97fd170bbc0f20dee9e6d63fe5e1a48077b29958545d4cf8f790aff2f2da6b27
..11c71fabb27ebef4f9b9f4a1c20b91b0,

bandersnatch_sha-512_ell2_ring - vector-6

04d3da92994eb327893f747b5aee14f82353ca88f909585b492ac6124c52da07,
6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
a99080c7bb9f8f4ccf68aa2cc8a946b03dbf07735564a5102009f2c8e5013bd2,
d52ea29fddc6c87f38e5d56ef2f5af434f90f28e8e122dd99284076b57d29997,
c8b1ce7a665f75506e6a64ef52aa7d8cbfecde0ad83946eb980ba4d06580f819,
635ccf83a424afe87946a36a532f2f2dd4e72b280f9a8d04e4abe3b7f3cbd89c,
3b15fefece933682d3a92dc57fdaa4da6e76f80c41cb926b00376576bced3668,
4b24cd4a4c25aa29aae4e969cf5be562cb9ea176593f8c06bb7343c8da71eac0,
3278c30b9d2fd950fa1a6e82e98375c8fd651f237806f8eaab44c0817b2cad10,
d59f47aa43f471c63ff9cebbab2aaea14feeeb914a3e1a9589350864f3d7b902,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..6be56f1a0c32af7ff857e278dc9a0dfeaf724b5961ab0a0147663f77d3e0dfb4
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
a3d604cc19e3ce06e7308730afb0637dc91f36160df749b86d9a8f21fd4608ba
..85c445bcba1951109771dcd3985f3e6ca88d087424d9dcb55ecce7ee85a2814b
..0263d0b9b22523c19efe00cf7038ccd4f9ba46212ca811dd272fbc0862c9fc92
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
b1d1af3da4bfabec5f057a969497db2d543ddbd950d3734cd3a335ca84766ae7
..59e1459b3a8b9ba4d9079c5dbe9bcaabb4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..83349c21fdf494a6185e79ae288e56f87d391254cfac6082a24cf8a351201fa8
..e042648b0c76ed86fe6eeb494e78d78daced9cb32c77769413d943dada87681c
..aee77b93b2555a690a1c26f2c60b9d6625314cd3c57b097d5998cf5babd25119
..b68688df48c1f2a13c86fecb38486a3318d8d3dcad6422b869306dde30465853
..77ecba05abdb9e34a910836787c7d40386aa13d380b8f4c122ceaa222ce83d3f

..82256cf293a6cbcd4fd25ee62c3f118ac321814e34eaf41f6465352c8f635f5c
..6b7fa5504b0f476fb8ddd03fad1d609f53654485e2277a228ac0625605363d6a
..02e79235e217776c51c1fb8fddd792d420432ad90b6d41558b73a9a0f1e22473
..e92b1bc2a5972704ec032489e09bf908f3c96c050f9dfbcff8b7a2a89bbba723
..a19706c0acbca700507fad16510996edf1cffaaddc9546d737c485d9b9671d2e
..8482bf4253e29dd0b85f5d45034dbd738f7b719479f7ac43b464c7593e7c9e63
..3a626c78cfeca7cc463c37ff1faa5d7bbac6bd50c4e3c70c5245949ea05fb696
..4161d144179e859d5b26556c2148c62db0d4cce4809c6e23ca82e2d96f80e47d
..6151c606a31740c4d4c03adf088315f07726408d2a83499790f0e475cd616463
..869a155965413f16f2ed89da328ccc4451faf86a0dd6933d13bef4809bd97903
..a6bc43dee6c599d001f2ebd1de959744,

bandersnatch_sha-512_ell2_ring - vector-7

9504efbeadf81b20a9cb64c1331915eb3718a574227458230d1d80dfa94e8b13,
704fd3784947de4db4fdcf0b477530d094bf5a656707b7d6cb43edbc4db7336b,
42616e646572736e6174636820766563746f72,
1f42,
832414b806b4dd6f98117818f1518667c8bb2bd625dfe353b1f2d22d4f32ead6,
1617b4fca3d7354fd43ae783bd6d0cfc526a80e257fcccfdf8e4d87e74dd7c6d,
6b279b85a1c55f55db80167e6bee997a5bf25c1767ffc2a34ae696baf606f5a0,
3a19e858c8f2cc41037737c43d23dbf3d124ea7059d30c6ab88aa184e14fe20c,
0a9e31ccc09e5464e53afa2057098da6208478f9334199c60bd4bc7443a3d5c5,
1ce7b8f6bcf3723106588842207b1199b8feaaf3bcc13874c0fa2e82a53b47b3,
7372c8e03008802e1bfdf4ac6b8832aded5b48ddb688af168cb67a882628ba22,
4b61c4d7d6179d8e1fe407a20e1ab22acec0a4834eb36fe03f254480d9d87e1b,
40d1a91ac8b287a2b754e9ba05c7c2091e140f0fb40753ebbd2573b6e9d1f514,
8b3031022897595ba3a280d6af047a46498f6fbfd62081d1df6a2e4b713014df
..bd7fcb7c0956648c043bc345a8fb3e1a2c73815bf87f6c3cb985a00c9e95e991
..307d3bef5e82f857b97f987da190e5fc6d77a9d24ac2068b701df1ab0487a1d8
..704fd3784947de4db4fdcf0b477530d094bf5a656707b7d6cb43edbc4db7336b
..72ce1e7899c633b92709702bdfbe74347d9bdcd1ad62b13f960ade3d1df899b3
..73154c7a701d5d6d38216c4be09800c024c35e4739e43f2b7aa5e0b55a281757
..a4cc78b2e3eb6a19a777ff20bb801c043f73d836aab81038e286560e260922b4
..0c5335e7855f8a38e9ffada8eadb2415f82b5822319fa53b66ef1c0469048ec3,
b6d67f50775000ca5f0b995b5f4700966876737294e494be8db5378e89fd93f0
..c91cfde506b1114d0e366b58368a478f9930089f630091dffabed54151764728
..798e46ab56a77f24e3ef58136b28eb8f7d4e34fa66e87f7a2eb68ec7d1445d3c
..96c1b168e2dcc743f9eadda76c041db42d39f27a58418f88c0ea67656a224934
..e12b5dfc8f0f460a95c2d467fa41907b,
aacb15136f83becdea7ff3f7f6e89f578f18693eb1017a6000ad08bb00c46d8a
..c7c5cfc8c04c327d118163472a999cc2b4fc7481cee0e755e488e0f66fd969f0
..43d6dbbf1eb6b4b1e5b80b3fbecd77866dbf841b4f3b569b422fbea2c744cadf
..a2f743398fc06ae65463e195772e22f830a1fa39776b22ae03821b92e59d798d
..45ff8df8b9d2c2f8129a14cf68fbc3528559091098dc6d09c6d2fbc56e79f865
..0fe55573d9ed6cf229fa5719622f5bc87b3feebb5edfa539120255c19e4c0132

```
..5b8d4ae6aa46bc3b58a7e3e694785d9e9b99147ba635e3d8f312cda6fd389858
..882820bc0ddbe1fa2f715258a12192cef40f1e721e3b22e9f1b70ec0de2efd18
..f2d7ccf8ec067750aaf94f0250818e986544e0673817694497c71b199e19d866
..0fd45bcdfba97ada27495f52751aab92f314eab7baccf8d860b927f24867e523
..994eeca7a78d92157e5a2951141cb5b5479db027f5c1fb309060b0dc2441472b
..27159c2b9e014d1ab0e0ae3ba229a57b4c5c45439f03142a86cef0d3aa1cde1d
..deb1ad05aae575da1b6b58d933b4dabd1a2fefe1562a8db459989d1b7988971a
..b024a1e137780b1b4dd2d60132772d8ddec617a426fe229e59fa94393c2d11b2
..7f597f7c35f5413d3d21303001f5915864a6e2670b134b051581bdf8ede11296
..add84f2b07e192eaf5657473ab4ce428b7f3bbf00449b4ba1b2683cb343c0f89
..ea8f4d7621a85156358011e3c1c48404c4e8f786fdc2b3078723a511ed14f1fc
..b471a445a2734188fc29db738137034a1b2b90c4d4eb444b062443eff4997441
..6827480f3894461c22fd527afe604668,
```

References

1. Internet Engineering Task Force *Verifiable Random Functions*; RFC Editor, 2023; https://datatracker.ietf.org/doc/rfc9381.

2. Burdges, J.; Ciobotaru, O.; Alper, H.K.; Stewart, A.; Vasilyev, S. Ring Verifiable Random Functions and Zero-Knowledge Continuations 2023; https://eprint.iacr.org/2023/002.

3. Ciobotaru, O.; Shirazi, F.; Stewart, A.; Vasilyev, S. Accountable Light Client Systems for PoS Blockchains 2022; https://eprint.iacr.org/2022/1205.

4. Masson, S.; Sanso, A.; Zhang, Z. Bandersnatch: A Fast Elliptic Curve Built over the BLS12-381 Scalar Field 2021; https://eprint.iacr.org/2021/1152.

5. Vasilyev, S.; Galassi, D. Ring Proof Technical Specification 2024; https://github.com/davxy/ring-proof-spec.

6. Internet Engineering Task Force *Hashing to Elliptic Curves*; RFC Editor, 2023; https://datatracker.ietf.org/doc/rfc9380.